

## Módulo 20 (18): Interoperabilidad y ecosistema mixto (opcional)

### Contenidos clave

**Contratos neutrales: OpenAPI y Protobuf:** Un contrato define cómo dos sistemas se comunican. El contrato permite que ambos lados sepan qué esperar.

- OpenAPI: es muy común en APIs REST. Define endpoints, métodos HTTP, parámetros, request, response y tipos de datos. FastAPI genera automáticamente un documento OpenAPI (<http://localhost:8000/docs>), es una representación interactiva del contrato de la API.
- Protobuf: (Protocol Buffers) es un formato de serialización y definición de contratos desarrollado por Google. En lugar de definir una API mediante JSON, se pueden definir los mensajes mediante un archivo .proto.

Diferentes lenguajes pueden utilizar ese mismo contrato, a eso se le llama contrato neutral.

**Servicios/clientes gRPC en Python:** gRPC es un framework para comunicación entre servicios. A partir del .proto se puede generar código para Python generando archivos llamados stubs que permiten utilizar el contrato desde Python sin tener que implementar manualmente todo el protocolo.

Ventajas:

- Contratos fuertes definidos
- Serialización eficiente
- Generación automática de clientes/stubs
- Muy útil para comunicación servicio-a-servicio

**Mensajería con RabbitMQ/Redis/Kafka:** Un servicio puede publicar un mensaje y otro consumirlo posteriormente, lo que permite desacoplar los servicios.

- RabbitMQ: es un broker de mensajes. Cuando se crea una orden se publica en RabbitMQ y los consumidores pueden procesarlo.
- Redis: es principalmente un almacén de datos en memoria, pero también proporciona mecanismos de mensajería, como Pub/Sub y Streams.
- Kafka: está diseñado para trabajar con flujos de eventos distribuidos y grandes volúmenes de mensajes. Es muy usado para arquitecturas orientadas a eventos y procesamiento de streams.

**Serialización: JSON/Avro/Protobuf:** serialización es convertir datos/objetos a una representación que pueda almacenarse o transmitirse.

- JSON: fácil de leer, muy utilizado e ideal para APIs REST, aunque, generalmente es más pesado que formatos binarios.
- Protobuf: se define y se serializa a formato binario, es compacto, rápido, tipado mediante el esquema y muy usado con gRPC.
- Avro: también utiliza esquemas para definir datos y streaming. Está muy orientado a datos y evolución de esquemas, mientras que Protobuf es muy común para RPC y contratos con servicios.

## Objetivos

### **- Definir contratos y comunicarse entre servicios heterogéneos.**

Se definió un contrato independiente del lenguaje mediante Protobuf, el contrato especifica el servicio, los mensajes y permite que otros servicios escritos en distintos lenguajes consuman el mismo contrato.

### **-Integrar mensajería para eventos.**

Se implementó un publicador Redis:

El evento está definido cómo:

```
7  @dataclass(frozen=True)
8  class OrderCreated:
9      order_id: int
10     customer_id: int
11     status: str
12     total: float
13
```

El publicador serializa el evento a JSON:

```
28     payload = json.dumps(asdict(event))
```

Y lo publica en el canal:

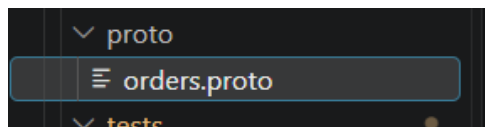
```
30     self._client.publish(
31         self._channel,
32         payload,
33     )
34
```

El canal utilizado es orders.created, esto permite desacoplar al productor del evento de los consumidores que quieran reaccionar a la creación de una orden.

## Laboratorio

### **- Definir .proto de Orders, generar stubs, servidor gRPC y cliente:**

El contrato se creó en:

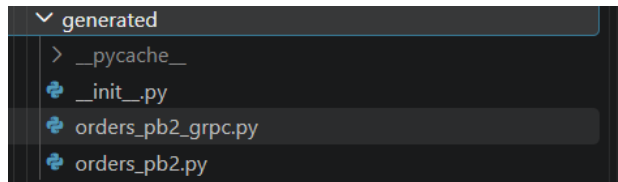


```

└─ proto
   └─ orders.proto
└─ tests
```

Este archivo funciona como fuente única del contrato de comunicación. Define: servicios gRPC, solicitudes, respuestas, tipos de datos, campos y números de identificación.

A partir del .proto se generaron los stubs:



- orders\_pb2.py contiene los mensajes Protobuf generados.
- orders\_pb2\_grpc contiene el stub del cliente, la clase base del servidor y la función de registro del servicio.

El servidor se implementó en grpc\_server.py, la clase implementa el servicio generado:

```
8 class OrdersService(orders_pb2_grpc.OrdersServiceServicer):
```

- Crear una orden:

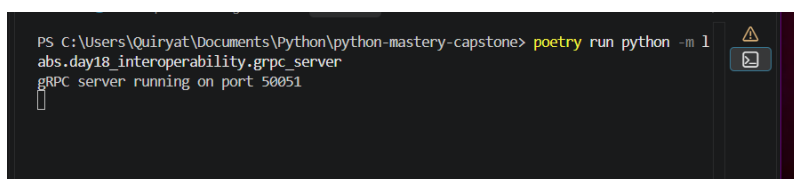
```
9 def CreateOrder(  
10     self,  
11     request,  
12     context,  
13 ):  
14     total = sum(item.quantity * item.unit_price for item in request.items)  
15  
16     return orders_pb2.OrderResponse(  
17         order_id=1,  
18         customer_id=request.customer_id,  
19         status="PENDING",  
20         total=total,  
21     )
```

El servidor recibe una solicitud Protobuf, calcula el total, construye una respuesta Protobuf y devuelve la respuesta al cliente.

- Obtener una orden:

```
23 def GetOrder(  
24     self,  
25     request,  
26     context,  
27 ):  
28     return orders_pb2.OrderResponse(  
29         order_id=request.order_id,  
30         customer_id=10,  
31         status="PENDING",  
32         total=100.0,  
33     )  
34
```

Arranque el servidor:



El cliente se implementó en grpc\_client.py, se conecta al servidor y crea el stub:

```
stub = orders_pb2_grpc.OrdersServiceStub(channel)
```

Y envía una solicitud:

```
15         return stub.CreateOrder(  
16             orders_pb2.CreateOrderRequest(  
17                 customer_id=10,  
18                 items=[  
19                     orders_pb2.OrderItem(  
20                         product_name="keyboard",  
21                         quantity=2,  
22                         unit_price=50.0,  
23                     )  
24                 ],  
25             )  
26         )  
27
```

El cliente recibe una respuesta interoperabilidad con:

```
PS poetry run python -m labs.day18_interoperability.grpc_client  
order_id: 1  
customer_id: 10  
status: "PENDING"  
total: 100  
  
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

#### **- Publicar evento OrderCreated en RabbitMQ/Redis:**

Mensajería con Redis: se inició Redis con Docker, se comprobó que estaba disponible, y se obtuvo una respuesta:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> docker run --rm --name
orders-redis -p 6379:6379 redis:7-alpine
Unable to find image 'redis:7-alpine' locally
7-alpine: Pulling from library/redis
4eb14bae0e1d: Pull complete
7eb50503c2b0: Pull complete
16333ee0c00f: Pull complete
83234986c742: Pull complete
1522f9c6f453: Pull complete
4f4fb700ef54: Pull complete
9928009b15c8: Pull complete
54779914f171: Download complete
3beebc06c4ca: Download complete
Digest: sha256:858f009f9709ce576febc734aa78b8f6d624b82571f9ddb6bda4377c833b3499
Status: Downloaded newer image for redis:7-alpine
1:C 30 Sep 2026 00:49:42.230 * oOoOoOoOoOoOo Redis is starting oOoOoOoOoOoOo
1:C 30 Sep 2026 00:49:42.230 * Redis version=7.4.11, bits=64, commit=00000000, modif
ied=0, pid=1, just started
1:C 30 Sep 2026 00:49:42.230 # Warning: no config file specified, using the default
config. In order to specify a config file use redis-server /path/to/redis.conf
1:M 30 Sep 2026 00:49:42.231 * monotonic clock: POSIX clock_gettime
1:M 30 Sep 2026 00:49:42.231 * Running mode=standalone, port=6379.
1:M 30 Sep 2026 00:49:42.232 * Server initialized
1:M 30 Sep 2026 00:49:42.232 * Ready to accept connections tcp
```

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> docker exec orders-red
is redis-cli ping
PONG
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

Se verificó la publicación usando un suscriptor: SUBSCRIBE orders.created y se publicó un evento:

```
15 def test_redis_publisher():
16     client = Mock()
17     publisher = RedisOrderPublisher(client)
18
19     event = OrderCreated(
20         order_id=1,
21         customer_id=10,
22         status="PENDING",
23         total=100.0,
24     )
```

El evento se serializó como JSON y se publicó en orders.created:

```
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run python -c "  
import redis; from labs.day18_interoperability.publisher import RedisOrderPublisher,  
    OrderCreated; client=redis.Redis(host='localhost', port=6379, decode_responses=True  
); publisher=RedisOrderPublisher(client); publisher.publish(OrderCreated(order_id=1,  
    customer_id=10, status='PENDING', total=100.0)); print('Evento publicado')"  
Evento publicado  
PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> █
```